Binary Addition

<u>Two 1-bit values</u>

A	B	A + B
0	0	0
0	1	1
1	0	1
1	1	10

"two"

Binary Addition

<u>Two *n*-bit values</u>

- Add individual bits
- Propagate carries
- E.g.,

```
 1     1
  10101        21
+ 11001      + 25
 101110        46
```

Binary Multiplication

<u>Binary, two 1-bit values</u>

A	B	A × B
0	0	0
0	1	0
1	0	0
1	1	1

Multiplication

<u>Binary, two *n*-bit values</u>

- As with decimal values
- E.g.,

```
     1110
   x 1011
  ------
     1110
    1110
   0000
  1110
 --------
 10011010
```

Complements

Complements are used in digital computers to simplify the subtraction operation and logical manipulation.

There are two types of complements for each base-r system:

1. The radix complement, r's complement.
2. The diminished radix complement, (r-1) complement.

Complements

When the value of the base r is substituted in the name , the two types are referred to as

The 2's complement and 1's is complement for binary numbers, and

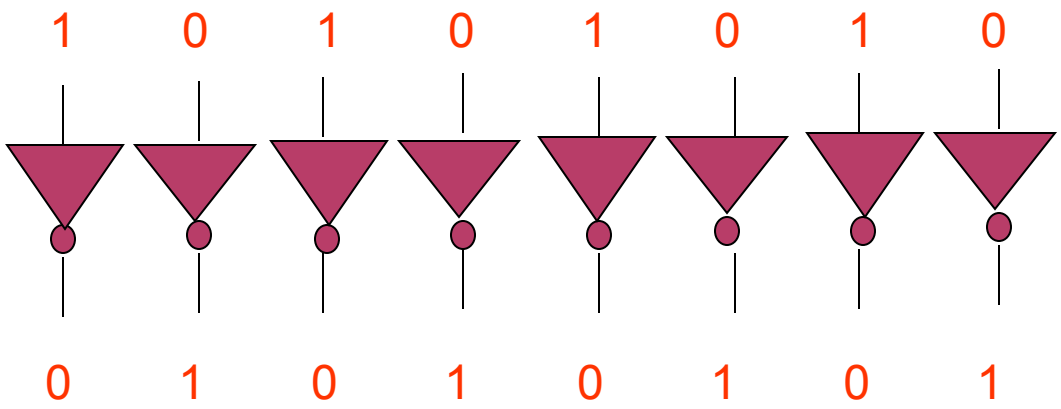
The 10's complement and 9's complement for decimal numbers.

Complements of Binary Numbers

- 1's complements
- 2's complements

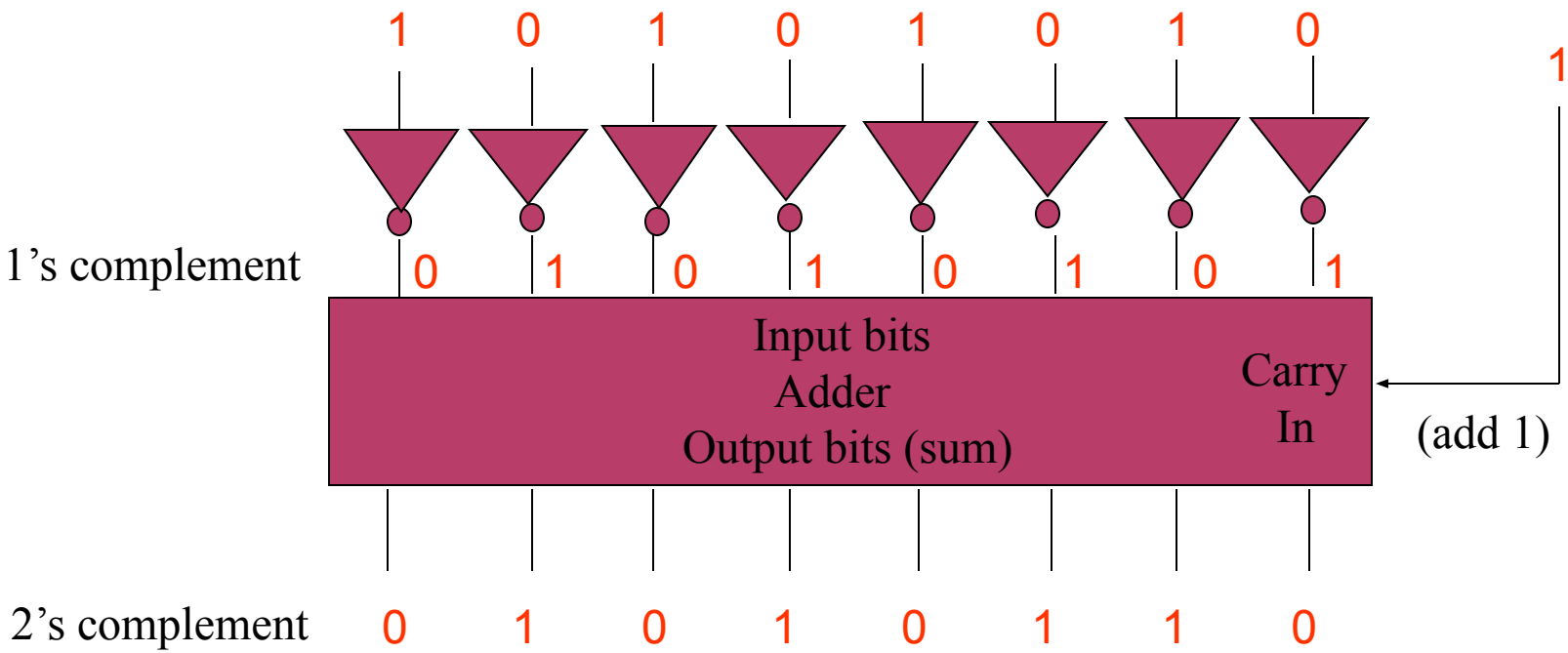
Complements of Binary Numbers

- 1's complement
- Change all 1s to 0s and all 0s to 1s



Complements of Binary Numbers

- 2's complement
- Find 1's complement and then add 1



A number N with radix r, r*'s* complement and *(r-1)'s* complement are defined as:

$$N^* = r\text{'s complement}$$
$$= r^n - N$$

$$N^{**} = (r-1)\text{'s complement}$$
$$= r^n - N - r^m$$

Where n and m are the numbers of integral and fractional digits of the number N respectively.

Examples

Find the 2's complement and 1's complement of N= 101101

$$n = 6,\ \ m = 0,\ N = 101101$$

2's complement $= N^* = 2^6 - 101101$

$1000000 \quad r^n = 2^6$

$-101101 \quad N$

$N^* = 010011 \quad r^n - N$

1's complement $= N^{**} = 2^6 - 101101 - 2^0$

$= N^* - 1$

$= 010010$

1's complement method

	1	1	0	1	1	=	27
-	1	0	0	1	0	=	18
	0	1	0	0	1	=	9

```
  1 1 0 1 1
+ 0 1 1 0 1   (1's complement of 10010 i.e., 01101)
1 0 1 0 0 0   (Carry)
+         1
  0 1 0 0 1
```

Therefore the result is 01001

	0	0	1	1	0	=	6
-	1	0	1	0	1	=	21
-	0	1	1	1	1	=	-15

```
  0 0 1 1 0
+ 0 1 0 1 0   (1's complement of 10101 i.e., 01010)
  1 0 0 0 0   ( No carry)
```

Therefore the result will be 1's complement of 10000 (01111) with a negative sign *i.e.*, Result is - 0 1 1 1 1

2's complement method

	1	1	0	1	1	=	27
-	1	0	0	1	0	=	18
	0	1	0	0	1	=	9

```
  1 1 0 1 1
+ 0 1 1 1 0   (2's complement of 10010 i.e., 01101+00001=01110)
1 0 1 0 0 1   (Carry)
+ 0 1 0 0 1
```

Therefore the result is + 0 1 0 0 1 (Positive)

	0	0	1	1	0	=	6
-	1	0	1	0	1	=	21
-	0	1	1	1	1	=	-15

```
  0 0 1 1 0
+ 0 1 0 1 1   (2's complement of 10101 i.e., 01010+00001=01011)
  1 0 0 0 1   ( No carry)
```

Since there is no carry, the answer is negative, and we have to take the 2's complement of the above to get the result.

2's complement of 10001= 0 1 1 1 0

```
                1
  0 1 1 1 1
```

Therefore the result is - 0 1 1 1 1

BINARY CODES

Digital systems represent and manipulate not only binary numbers, but also many other discrete elements of information. Any discrete element of information that is distinct among a group of quantities can be represented with a binary code. For example:

Decimal 396 is represented in Binary-Coded Decimal (BCD) with 12 bits as 0011 1001 0110, with each group of 4 bits representing one decimal digit.

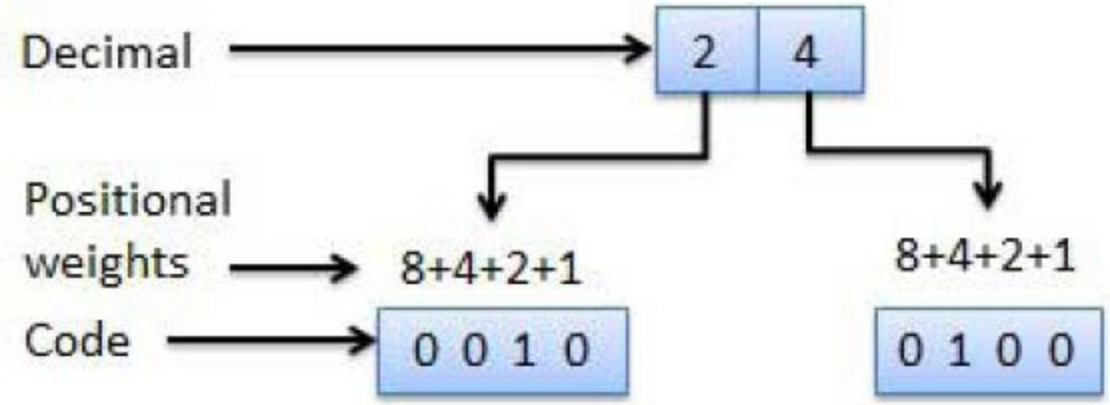
Advantages of Binary Code

- Binary codes are suitable for the computer applications.
- Binary codes are suitable for the digital communications.
- Binary codes make the analysis and designing of digital circuits if we use the binary codes.
- Since only 0 & 1 are being used, implementation becomes easy.

Different Types of Binary Codes

Weighted Codes:

Weighted binary codes are those binary codes which obey the positional weight principle. Each position of the number represents a specific weight. Several systems of the codes are used to express the decimal digits 0 through 9. In these codes each decimal digit is represented by a group of four bits.



Non-Weighted Codes:

In this type of binary codes, the positional weights are not assigned. The examples of non-weighted codes are Excess-3 code and Gray code.

Excess-3 code

The Excess-3 code is also called as XS-3 code. The excess-3 codes are obtained as follows −

$$\text{Decimal Number} \longrightarrow \text{8421 BCD} \xrightarrow[0011]{\text{Add}} \text{Excess-3}$$

Decimal	BCD 8 4 2 1	Excess-3 BCD + 0011
0	0 0 0 0	0 0 1 1
1	0 0 0 1	0 1 0 0
2	0 0 1 0	0 1 0 1
3	0 0 1 1	0 1 1 0
4	0 1 0 0	0 1 1 1
5	0 1 0 1	1 0 0 0
6	0 1 1 0	1 0 0 1
7	0 1 1 1	1 0 1 0
8	1 0 0 0	1 0 1 1
9	1 0 0 1	1 1 0 0

Four Different Binary Codes for the Decimal Digits

Decimal Digit	BCD 8421	2421	Excess-3	8,4,-2,-1
0	0000	0000	0011	0000
1	0001	0001	0100	0111
2	0010	0010	0101	0110
3	0011	0011	0110	0101
4	0100	0100	0111	0100
5	0101	1011	1000	1011
6	0110	1100	1001	1010
7	0111	1101	1010	1001
8	1000	1110	1011	1000
9	1001	1111	1100	1111

Digit 2421 = 2x1+4x1+2x0+1x1=2+4+0+1=7

Excess-3 = 0111+0011=1010

Digit 8,4,-2,-1 = 8x1+4x0+(-2x0)+(-1x1)=8+0+0-1=7

The advantages of the Gray code over the straight binary number sequence is that only one bit is the code group changes in going from one number to the next. For example-

In going from 7 to 8, the Gray code changes from 0100 to 1100. only the first bit changes, from 0 to 1; the other three bits remain the same. By contrast, with binary numbers the change from 7 to 8 will be from 0111 to 1000, which causes all four bits to change values.

Gray Code	Decimal Equi.
0000	0
0001	1
0011	2
0010	3
0110	4
0111	5
0101	6
0100	7
1100	8
1101	9
1111	10
1110	11
1010	12
1011	13
1001	14
1000	15

ASCII Character Code

American Standard Code for Information Interchange (ASCII). It is a seven bit code, but most of the computers manipulate an eight bit quantity as a single unit called a byte. Therefore, ASCII characters most often are stored one per byte. The extra bit is sometimes used for other purposes, depending on the application.

10 decimal digits, 26 letters of the alphabet (Upper and Lowercase), and a number of special characters. Also 34 control characters are designated in the ASCII table with abbreviated names.

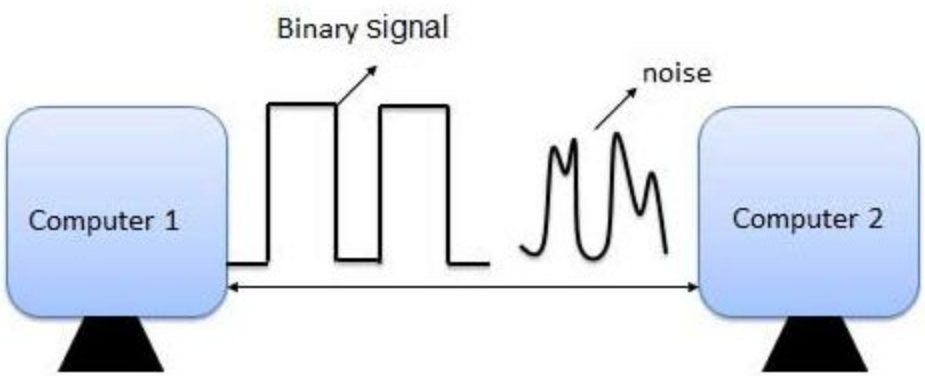
American Standard Code for Information Interchange (ASCII).

USASCII code chart

b_7 b_6 b_5 →					0 0 0	0 0 1	0 1 0	0 1 1	1 0 0	1 0 1	1 1 0	1 1 1
Bits b_4	b_3	b_2	b_1	Column → / Row ↓	0	1	2	3	4	5	6	7
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	X	h	x
1	0	0	1	9	HT	EM	)	9	I	Y	i	y
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
1	0	1	1	11	VT	ESC	+	;	K	[	k	{
1	1	0	0	12	FF	FS	,	<	L	\	l	|
1	1	0	1	13	CR	GS	–	=	M	]	m	}
1	1	1	0	14	SO	RS	.	>	N	^	n	~
1	1	1	1	15	SI	US	/	?	O	_	o	DEL

What is Error

Error is a condition when the output information does not match with the input information. During transmission, digital signals suffer from noise that can introduce errors in the binary bits travelling from one system to other. That means a 0 bit may change to 1 or a 1 bit may change to 0.



Error-Detecting codes

Whenever a message is transmitted, it may get scrambled by noise or data may get corrupted. To avoid this, we use error-detecting codes which are additional data added to a given digital message to help us detect if an error occurred during transmission of the message. A simple example of error-detecting code is **parity check**.

How to Detect and Correct Errors?

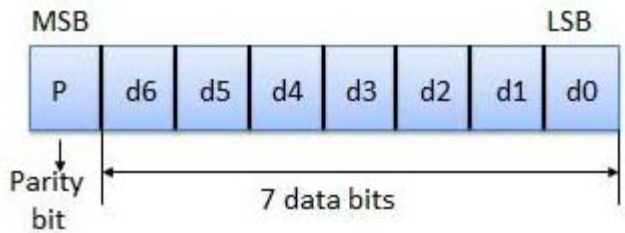
To detect and correct the errors, additional bits are added to the data bits at the time of transmission.

The additional bits are called **parity bits**. They allow detection or correction of the errors.

The data bits along with the parity bits form a **code word**.

Parity Checking of Error Detection

It is the simplest technique for detecting and correcting errors. The MSB of an 8-bits word is used as the parity bit and the remaining 7 bits are used as data or message bits. The parity of 8-bits transmitted word can be either even parity or odd parity.



- **Even parity** -- Even parity means the number of 1's in the given word including the parity bit should be even (2,4,6,....).
- **Odd parity** -- Odd parity means the number of 1's in the given word including the parity bit should be odd (1,3,5,....).

Use of Parity Bit

The parity bit can be set to 0 and 1 depending on the type of the parity required.

- For even parity, this bit is set to 1 or 0 such that the no. of "1 bits" in the entire word is even. Shown in fig. (a).
- For odd parity, this bit is set to 1 or 0 such that the no. of "1 bits" in the entire word is odd. Shown in fig. (b).

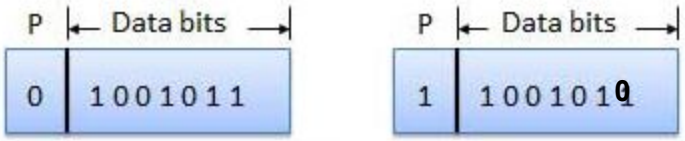


Fig. (a)

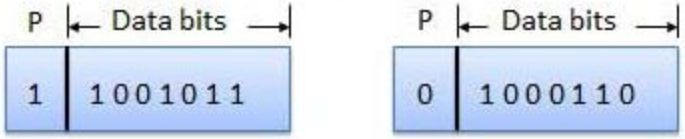
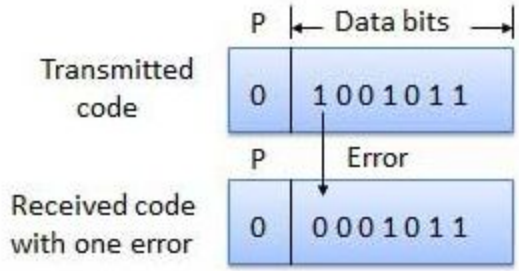


Fig. (b)

How Does Error Detection Take Place?

Parity checking at the receiver can detect the presence of an error if the parity of the receiver signal is different from the expected parity. That means, if it is known that the parity of the transmitted signal is always going to be "even" and if the received signal has an odd parity, then the receiver can conclude that the received signal is not correct. If an error is detected, then the receiver will ignore the received byte and request for retransmission of the same byte to the transmitter.